

System Design Notes: WebSockets vs. Socket.IO

SDE Interview Prep Guide

1 Foundations of Real-Time Communication

Traditional HTTP is stateless and unidirectional (Client → Server). Real-time systems require bidirectional data flow.

- **Standard Polling:** Client repeatedly requests data. High overhead, wastes bandwidth on empty responses.
- **Long Polling:** Server holds the client's request open until data is available. Better, but still incurs connection teardown/rebuild costs.
- **Server-Sent Events (SSE):** Persistent one-way stream (Server → Client). Ideal for live feeds, but cannot receive client data over the same channel.
- **WebSockets:** A persistent, bidirectional, full-duplex TCP stream (RFC 6455).

2 Core WebSocket Mechanics

2.1 The HTTP Handshake

WebSockets start as a standard HTTP/1.1 request using the **Upgrade** header to bypass firewalls.

```
GET /chat HTTP/1.1
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
```

The server responds with an HTTP 101 **Switching Protocols** status code, abandoning HTTP for a raw TCP stream.

2.2 Framing & Connection Management

Data is sent in lightweight binary or text **frames**. To prevent load balancers (e.g., AWS ALB, NGINX) from killing idle connections, WebSockets use a Heartbeat mechanism:

- **Ping (0x9):** Server sends a control frame.
- **Pong (0xA):** Client must immediately return the same payload.

3 Socket.IO Abstractions

Socket.IO is a high-level library built on top of WebSockets (via Engine.IO) providing features critical for production systems.

- **Fallback Mechanism:** Starts with HTTP Long Polling to guarantee a connection, then seamlessly upgrades to WebSockets in the background.
- **Auto-Reconnection & Buffering:** Automatically handles exponential backoff for disconnected clients and buffers offline messages to prevent data loss.
- **Multiplexing:** Uses **Namespaces** (logic pools sharing one TCP connection) and **Rooms** (ephemeral sub-channels for broadcasting, e.g., chat lobbies).

4 System Design & Scaling

Scaling stateful WebSockets horizontally across multiple server nodes introduces two major architectural challenges.

4.1 Sticky Sessions (Session Affinity)

Because Socket.IO utilizes a multi-request handshake (Long Polling → Upgrade), a standard round-robin load balancer will break the connection. The load balancer must be configured with **Sticky Sessions** (using a cookie hash) to route all requests from a specific client to the exact same backend server.

4.2 The Pub/Sub Backplane (Redis Adapter)

If Client A is on Server 1, and Client B is on Server 2, they cannot communicate directly. Servers must be decoupled using a Redis Adapter. When Server 1 receives a message, it publishes it to Redis. Server 2 subscribes to Redis, receives the message, and emits it to Client B.

5 Socket.IO API Reference

Understanding the scope of emit functions is a common interview requirement.

```
1 const io = require('socket.io')(3000);
2
3 io.on('connection', (socket) => {
4
5     // 1. socket.emit: Sends ONLY to this specific client
6     socket.emit('private_message', 'Welcome to the server!');
7
8     // 2. socket.broadcast.emit: Sends to everyone EXCEPT this client
9     socket.broadcast.emit('alert', `User ${socket.id} joined.`);
10
11     socket.on('send_message', (msg) => {
12         // 3. io.emit: Sends to EVERYONE globally
13         io.emit('chat_update', {
14             sender: socket.id,
```

```
15         text: msg
16     });
17 });
18 });
```

Listing 1: Socket.IO Event Scopes

```
1 io.on("connection", (socket) => {
2     // Join a specific match lobby
3     socket.on("join_match", (matchId) => {
4         socket.join(matchId);
5     });
6
7     // Broadcast only to users in that specific lobby
8     socket.on("draw_line", (data) => {
9         socket.to(data.matchId).emit("new_line", data);
10    });
11 });
```

Listing 2: Rooms and Multiplexing